DST 30 min — Scripting Python

Durée : 30 minutes · Sans ordinateur · Documents : aide-mémoire en fin de sujet uniquement

Consignes : entourer la/les bonnes réponses pour le QCM. Réponses rédigées et code directement sur la copie, de façon lisible. Barème indicatif : Partie 1 — 6 pts, Partie 2 — 6 pts, Partie 3 — 8 pts.

Partie 1 — QCM (entourer la/les bonnes réponses)

Q1. La Disponibilité, c'est :
A. restreindre l'accès aux données
B. garantir que le service est opérationnel
C. garantir que les données n'ont pas été modifiées
D. tracer qui a fait quoi

Q2. `Accounting` (traçabilité), c'est :
A. vérifier login + MFA
B. vérifier les droits d'un utilisateur
C. enregistrer qui a fait quoi via les logs
D. chiffrer les mots de passe

Q3. Un `IOC` est :
A. un code qui exploite une faille
B. un indice de compromission (IP, hash, log suspect)
C. une donnée volée exfiltrée
D. une phase de la Kill Chain

Q4. Le scan de ports correspond à quelle phase de la Kill Chain ?
A. Exfiltration
B. Exploitation
C. Reconnaissance
D. Installation C2

Q5. Quelle affirmation est vraie ?
A. Python est compilé
B. Python est interprété
C. une variable ne peut jamais changer de type
D. `None` signifie une erreur fatale

Q6. Quel est le bon réflexe avant `pip install` ?
A. toujours faire `sudo pip install` pour éviter les erreurs de droits
B. vérifier nom exact, auteur, pages PyPI/GitHub ; pinner + auditer (`pip audit`)
C. installer la version la plus haute quel que soit le registre
D. installer depuis un nom approximatif, c'est plus rapide

Partie 2 — Questions

Q7. À quoi sert le `timeout` sur une requête réseau ? Que risque-t-il de se passer si on n'en met pas ?

Q8. Dans le contexte d'un appel HTTP, quel(s) élément(s) retrouve-t-on à la fois dans la requête et la réponse ?

Q9. Expliquer comment une librairie open-source peut être « weaponized » (piégée pour attaquer) : par quels vecteurs un attaquant peut-il piéger une librairie ? À quel(s) moment(s) le code malveillant s'exécute-t-il chez la victime ? Quel est l'impact potentiel ?

Partie 3 — Écriture de code sur feuille

Exercice 1 — Compréhensions list / dict

Chaque question demande **une seule expression de compréhension**.

1a. *List comprehension* : renvoyer les IP commençant par `"1."`. Exemple :

```
ips: list[str] = ["1.2.3.4", "5.6.7.8", "1.2.3.4", "9.9.9.9", "1.2.3.4"]
```

Valeur attendue : `["1.2.3.4", "1.2.3.4", "1.2.3.4"]`.

1b. *List comprehension* : renvoyer le carré de chaque nombre. Exemple :

```
nombres: list[int] = [1, 2, 3, 4, 5]
```

Valeur attendue : `[1, 4, 9, 16, 25]`.

1c. *Dict comprehension* : inverser `ports` en `{port: nom}`. Exemple :

```
ports: dict[str, int] = {"ssh": 22, "http": 80, "https": 443}
```

Valeur attendue : `{22: "ssh", 80: "http", 443: "https"}`.

Exercice 2 — filtrage de logs

Écrire une fonction `compter_ips(lignes: list[str]) -> dict[str, int]` qui :

- renvoie un dict `{ip: nombre d'échecs}`, où nombre d'échecs correspond au nombre de lignes où « failed » apparaît (pour une IP donnée).

Exemple attendu :

```
lignes: list[str] = [
    "ok admin 10.0.0.1",
    "failed root 10.0.0.5",
    "failed guest 10.0.0.5",
    "ok admin 10.0.0.1",
]
```

`compter_ips(lignes)` ⟶ `{"10.0.0.5": 2}`.

Rappel des opérations Python

Aide-mémoire fourni avec le sujet (Modules 2 à 5).

Opération	Exemple / usage
Test d'appartenance (sous-chaîne)	`"failed" in ligne` ⟶ `True`/`False`
Découper une chaîne	`s.split()` ⟶ découpe sur les espaces ; `s.split(".")` ⟶ découpe sur les `.` par ex `"failed root 10.0.0.5".split()` ⟶ `["failed", "root", "10.0.0.5"]`
Dernier élément	`seq[-1]` ⟶ dernier élément (ex. IP = `ligne.split()[-1]`)
Tester un préfixe	`s.startswith("ab")` : renvoie `True` si `s` commence par `ab`
Boucler	`for elem in sequence:`
List comprehension	`[expr for x in seq if cond]`
Dict comprehension	`{k: v for ...}`
Parcourir un dict	`d.items()` ⟶ liste des paires (clé, valeur) ex. `for k, v in d.items():`
Puissance	`n ** 2`
Longueur	`len(seq)` ⟶ nombre d'éléments / caractères (ex. `len("1.2.3.4") == 7`)